

Deep learning and representation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 2, `nw/semantic`: a transformer encoder fine-tuned with LoRA for tags and priority, exported to ONNX with parity proof and an int8 graph, benchmarked against Project 1, and put through the same loop: a gate that reads three files, a six-step pipeline, a registry version you approve, serving from the registry, and a retraining run with a tightened bar.

Attention replaced recurrence in 2017, and today's models are built on it

Neural networks

Layers learn their own features from raw input. Project 1's TF-IDF features were designed by hand.

2017, the transformer

Attention alone, no recurrence: a new best on translation after 3.5 days on eight GPUs.

2018, BERT

An encoder: pretrain on unlabelled text, then fine-tune with one extra output layer.

Decoders, the LLMs

The same block trained to predict the next token. Generation, and Project 3.

Project 2 is the third card: a pretrained encoder, fine-tuned for Northwind's tags and priority. It reads a ticket; it does not write one.

Prompt first; fine-tune when the task is narrow, frequent and yours

Approach	What changes	Who pays for it	Here
Pretraining	every weight, from vast unlabelled text	a lab, once	you download it
Fine-tuning	weights, for one task, from your labels	you, per task	Project 2
Prompting	nothing in the model: instructions and examples	you, per call	Project 3

Fine-tune when you have more examples than fit in a prompt, need lower cost and latency at scale, hold private training data, or want a smaller model to do one task well. Otherwise, with evals in place, a prompt may be all you need.

LoRA trains a sliver of the weights, and learns less but forgets less

10,000x

fewer trainable parameters than full fine-tuning of a 175B model, with 3x less GPU memory

- Freeze the pretrained weights; train small low-rank matrices beside them: adapters
- QLoRA fine-tunes a 65B model on one 48 GB GPU with 4-bit base weights
- The trade: LoRA learns the new task less well than full fine-tuning, and keeps more of what the model already knew
- Project 2 puts adapters on the encoder's attention and trains the two heads

Distillation and quantization make a model cheap enough to serve

Distillation: a small student learns from a big teacher

- DistilBERT: 40 percent smaller, 60 percent faster, 97 percent of BERT's language understanding. Project 2's base
- DeepSeek distilled its reasoning model into open models from 1.5B to 70B; the 32B one beat o1-mini on AIME 2024

Quantization: fewer bits per weight

- int8 halves inference memory with no loss, up to 175B
- 4-bit methods (GPTQ, AWQ) run over 3x faster than FP16
- ONNX Runtime recommends dynamic int8 for transformers: Project 2's export

The growth in 2026 is in small, open and on-device models

A prediction

Gartner predicts that by 2027 organizations will use small, task-specific models three times more than general-purpose LLMs.

On the phone

Apple gives developers an on-device model of about 3B parameters at 2 bits per weight. Its 2026 family adds a 20B sparse on-device model.

Open and at the edge

Google's Gemma 4, Apache 2.0, ships edge models; Google says the smallest runs in under 1.5 GB of memory on some devices.

Big models set the frontier. Small ones do most of the work, closer to the data and at a price per call you can plan.

Today you fine-tune a small encoder and hold it to Project 1's bar

PART 1

Foundations and the AI service layer

PART 2

Machine learning in production

PART 3

Deep learning and representation

PART 4

Generative AI: retrieval and evaluation

PART 5

Agentic AI systems

PART 6

Deployment, observability and capstone

Project 1 orders the queue and cannot say what a ticket is about. Project 2 tags it: DistilBERT with LoRA adapters, exported to ONNX, quantized to int8, benchmarked against Project 1 on the same test split, and gated by the same loop.

The deliverable is a benchmark table and a gate decision on it, not a personal best on F1.

Today, step by step

- 1 Device, memory, the data contract
- 2 Read the loop and see the tests fail
- 3 Freeze the encoder, inject LoRA
- 4 The loss and the training step
- 5 Per-tag thresholds, then train
- 6 Track and compare runs
- 7 Checkpoints that resume **OPTIONAL**
- 8 Export, quantise, prove parity
- 9 The benchmark table
- 10 The gate moves latest; the card
- 11 The semantic pipeline
- 12 Approve and serve from the registry
- 13 Service, third drift signal **OPTIONAL**
- 14 Backtest the next version **OPTIONAL**
- 15 Retrain with a tightened bar **OPTIONAL**
- 16 German error analysis **OPTIONAL**

The tests download nothing. Step 2 prints `2 failed, 9 passed, 21 errors` .

"It does not fit" is usually the optimiser, not the model

Held per parameter, fp32 with Adam	Bytes
Weights	4
Gradients	4
Two optimiser moments	8
Full fine-tuning	16

about 1 GB

66 million DistilBERT parameters at 16 bytes each, before a single activation

Activations scale with batch size times sequence length times hidden size times layers. Write the number down before you touch a model.

Freeze the encoder and train under one percent of it

```
for p in model.encoder.parameters():  
    p.requires_grad = False  
cfg = LoraConfig(r=8, lora_alpha=16,  
                lora_dropout=0.05,  
                target_modules=["q_lin", "v_lin"],  
                bias="none")  
model.encoder = get_peft_model(model.encoder, cfg)
```

0.29%

192,058 of 66,554,938 parameters train: the adapters on query and value, and the two heads

Multi-label is not multi-class

	Tags	Priority
Can two be true at once?	Yes: "SSO" and "Outage"	No
Loss	binary cross - entropy per tag	cross - entropy with class weights
Decision	one threshold per tag, chosen on validation	argmax
Wrong choice looks like	softmax across tags: exactly one fires	a flat 0.5 under - predicts every rare tag

Priority here has no threshold rule: this head does not protect P0, and the benchmark will show it.

Training ends with a candidate, not a decision

0.742

P0 recall on the `test:` line. 0.839 on the last validation line. Same model, same argmax

- 2,000 ticket subset, six epochs, 102 s on Apple silicon
- Test: tag micro-F1 0.541, tag macro-F1 0.609, priority macro-F1 0.693
- `candidate <version> registered, not promoted` : this gate reads parity and the int8 row, which do not exist yet
- `make runs-semantic` : one row, the same numbers, the seconds and the device

ONNX is the serving boundary

- 1 Named inputs and outputs, dynamic axes on batch and sequence, opset 17, `dynamo=False`
- 2 Parity on 20 held-out tickets: fp32 max logit difference $1.0e-5$; the gate holds it to $1e-4$
- 3 Dynamic int8: 265.7 MB to 66.8 MB, ratio 3.98, judged by task metrics, not logits

Where ONNX leaks

Custom operators, data-dependent control flow, anything the tracer turns into a constant.

If parity fails only on the second ticket of a batch, the axes are fixed.

Three points of macro-F1 cost fifteen times the latency

Model	Priority macro - F1	P0 recall	Tag micro - F1	p50 ms	p95 ms	Size MB
Project 1: TF-IDF and logistic regression	0.663	0.903	n/a	1.7	2.3	6.8
Project 2: PyTorch fp32	0.693	0.742	0.541	28.0	53.3	268.0
Project 2: ONNX fp32	0.693	0.742	0.541	35.1	57.7	265.7
Project 2: ONNX int8	0.682	0.677	0.522	12.5	18.2	66.8

Six epochs, Apple silicon, test split, batch size one. Remeasured with the locked dependencies the int8 row reads 0.666, 0.645, 0.524 at 9.6 and 13.5 ms; the gate quotes that run. Your latencies are your machine's.

A candidate is served only by passing a gate that reads three files

Bar	Value	Yours
Tag micro-F1, test	at least 0.50	0.541
P0 recall, test	at least 0.70	0.742
fp32 ONNX parity	at most 1e-4	1.0e-5
int8 against fp32, either head	drop at most 0.03	0.027, 0.017
int8 p95 latency	at most 100 ms	13.5
Against production, both heads	drop at most 0.02	0.000, 0.000
Same test split	identical data hash	308ad1151d5c

`metadata.json`, `export_report.json`, `benchmark.json`. A missing one is named, not guessed.

The pipeline puts three steps between training and the gate

Step	Adds
data prep	Project 1's contract, then the tag checks
train	the candidate, as <code>make train-semantic</code> makes it
export	the graphs, int8, parity
benchmark	<code>benchmark.json</code> , training a Project 1 reference if none is at hand
gate	the decision, <code>passed_int</code> for SageMaker's ConditionStep
register	the candidate, only if the gate passed

Local runner, six epochs, measured

About four minutes on Apple silicon. The step 5 test line to the digit, parity 1.00e-05, `PROMOTED` , `registered northwind-solo-semantic version 1 (candidate)` , `SUCCESS` .

The cloud instances have no accelerator: many times longer. Submit and go on.

Approve the int8 numbers, because int8 is what gets served

The registry version carries the int8 metrics beside the fp32 ones: `int8_tag_micro_f1` 0.524, `int8_priority_macro_f1` 0.666, `int8_p95_ms` 20.5 on the author's laptop.

The reason names the tag score, the int8 drop and the German row.

AWS

The approval event creates `northwind-<handle>-semantic` on Serverless Inference; solo mode deploys directly.

Google Cloud and Local

The course service reads `NW_MODEL_URI`: a new Cloud Run revision, or `models:/northwind-solo-semantic/1` on :8002.

The tag rate says the model is firing on noise

4.15

tag rate PSI after fifty forty-character tickets. Text length 8.87, as in Project 1; priority 5.08; level alert

- Text length and priority reuse Project 1's monitor unchanged
- The third signal: the share of requests carrying each tag against the training rows' `tag_share`
- A forty-character ticket fires fifteen to twenty of the 54 tags: the ten lowest thresholds from the card at work
- The same `drift_alert` line the capstone alarms on

The column you sort on is not the model

Two epochs, 35 seconds: test P0 recall 0.806, higher than the served 0.742. The gate before the export refuses to run: `export_report.json` is missing.

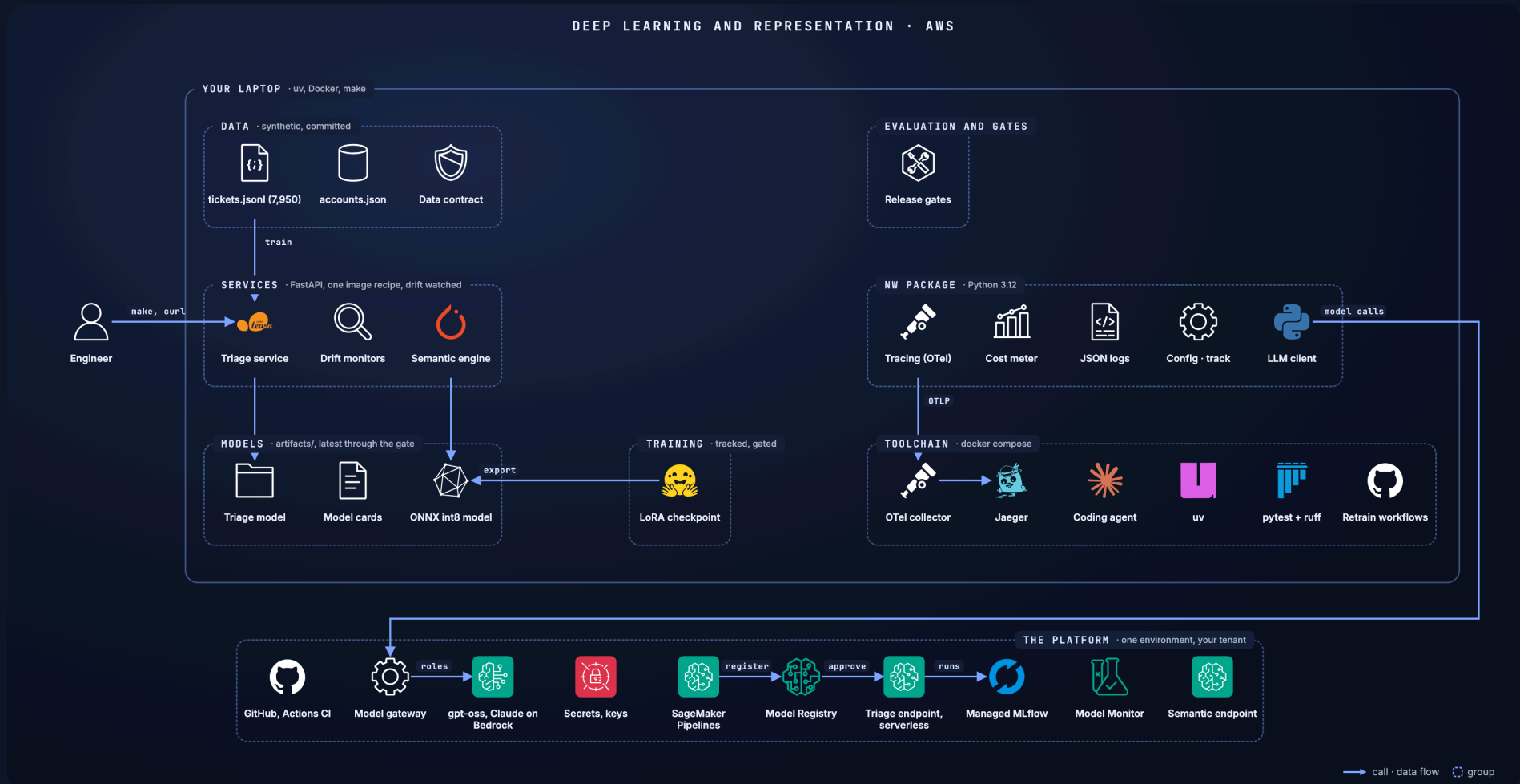
Backtest on 794 tickets: priority agreement 0.728, tag set agreement 0.035. `GATE FAILED` on tags and both relative bars.

Retrain with a tightened bar

`--min-tag-micro-f1 0.55` is a pipeline parameter, not a code change. The run ends `FAILURE` at the register step; nothing reaches the registry.

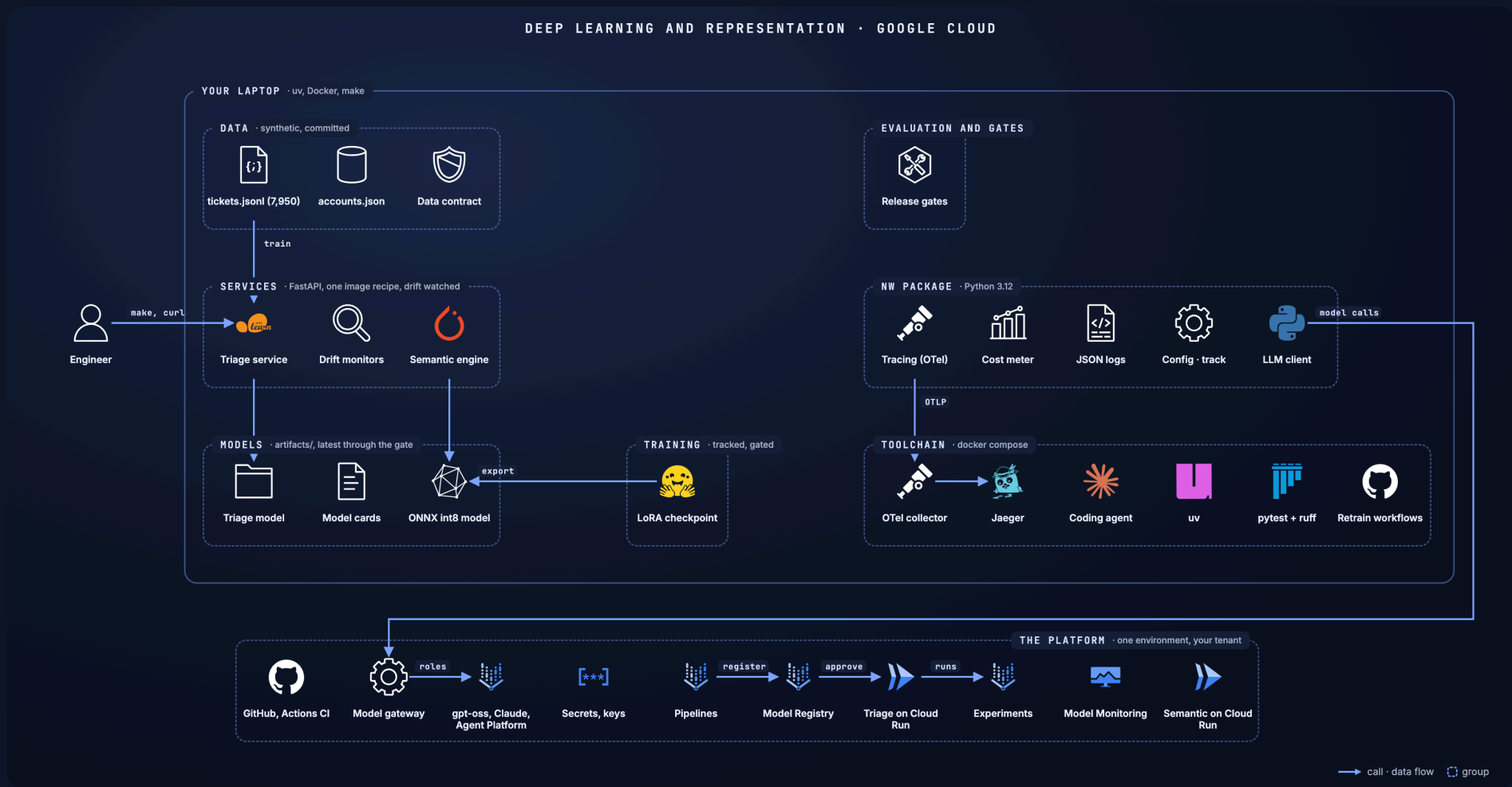
`retrain-semantic.yml` is on demand only: a hosted CPU runner needs 10 to 20 minutes for two epochs.

What exists after today, AWS track



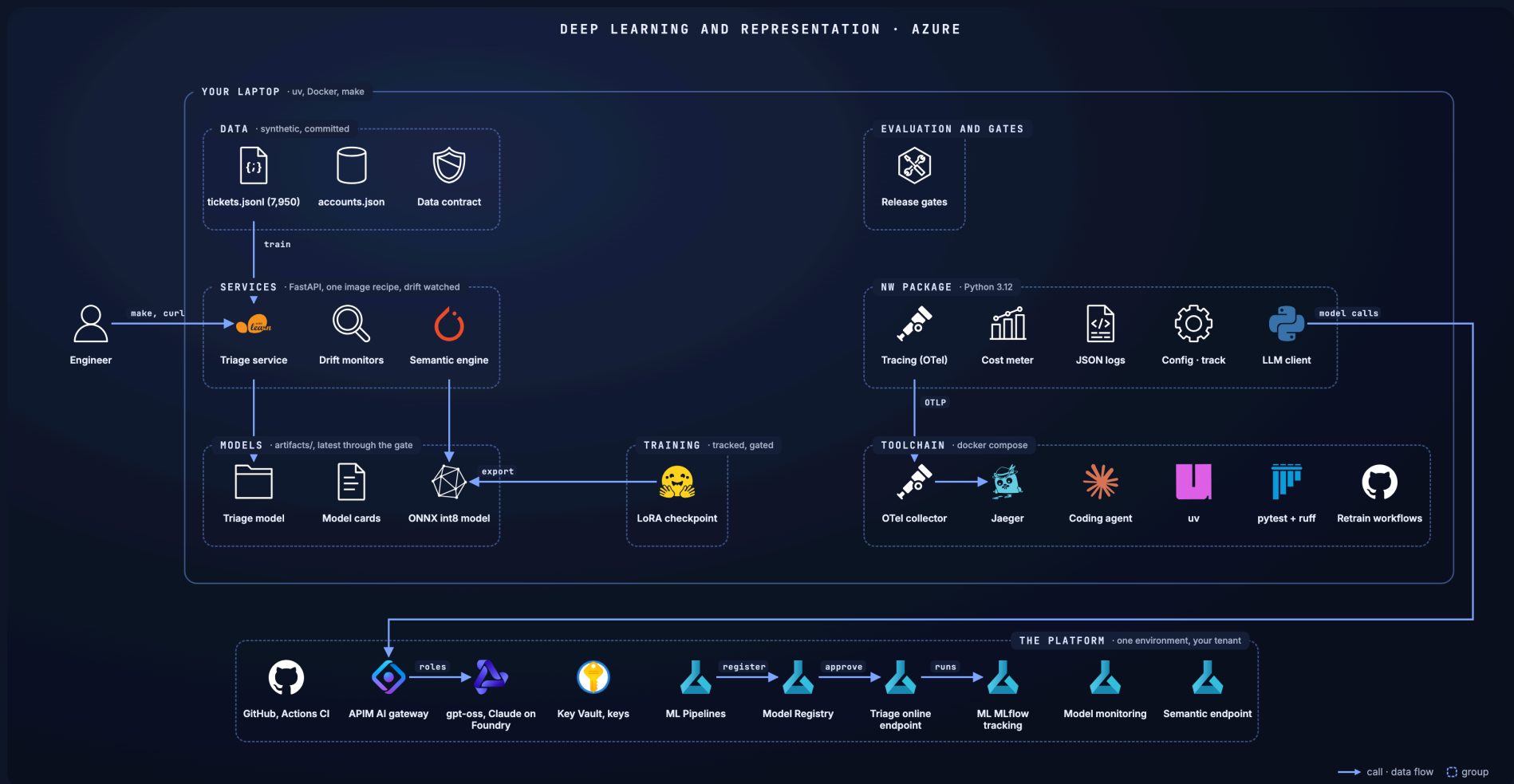
The semantic engine beside the triage service, the ONNX int8 model from the LoRA checkpoint, and on the platform a second pipeline, registry entry and serverless endpoint.

What exists after today, Google Cloud track



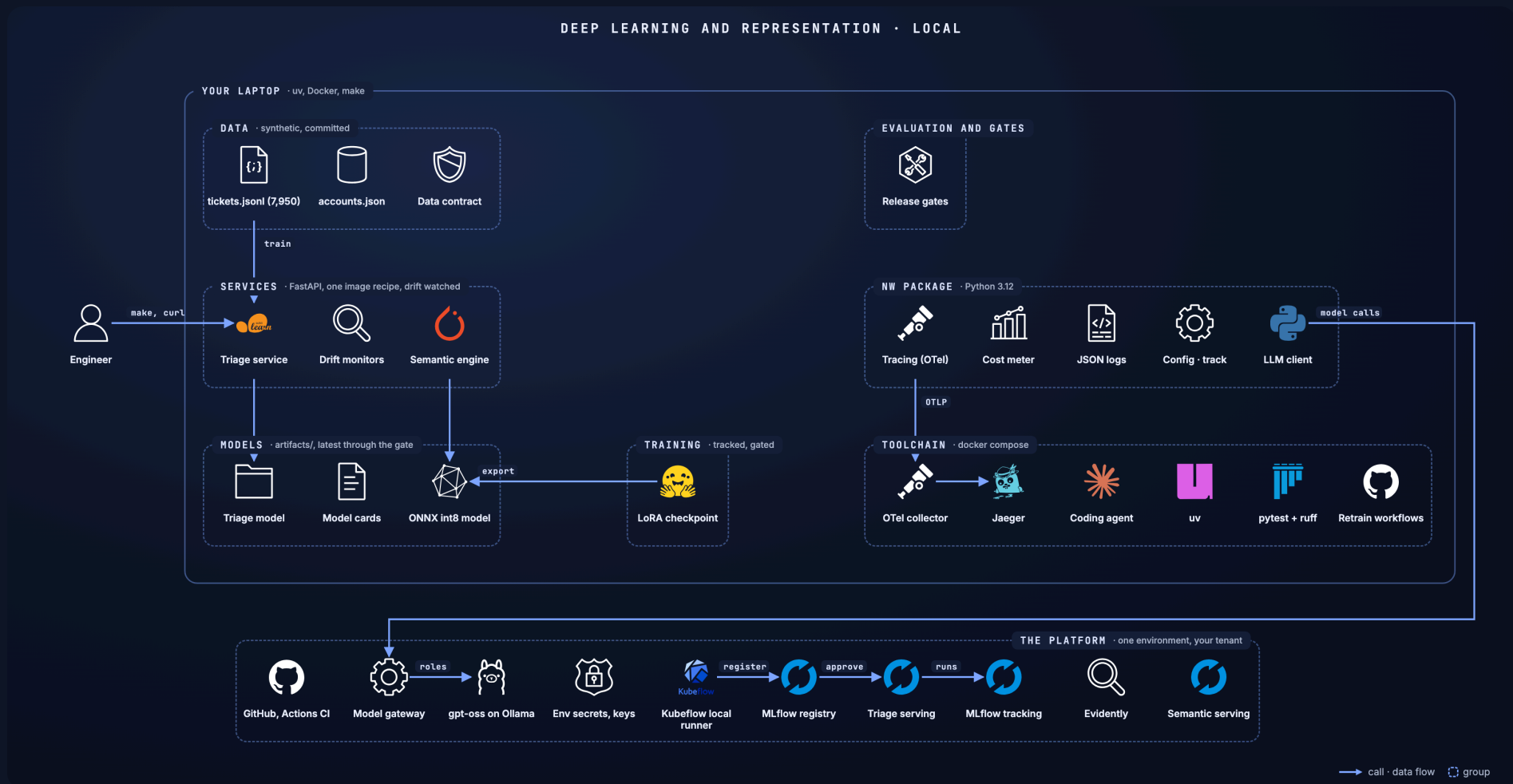
Same engine, same graph; a second pipeline and registry entry, and your Cloud Run semantic service reading the approved version.

What exists after today, Azure track



Same engine, same graph; a second Azure Machine Learning pipeline and registry entry, and your semantic managed online endpoint serving the approved version.

What exists after today, Local track



Same engine, same graph; the Kubeflow local runner, the MLflow registry, and semantic serving from the registry's artifact.

Same loop, four platforms

Component	AWS	Google Cloud	Azure	Local
Pipelines	SageMaker Pipelines	Pipelines, the Local YAML	Azure ML pipelines, serverless	Kubeflow local runner
Registry	package group - <code>semantic</code>	Model Registry, aliases	Azure ML registry, <code>stage</code> tag	MLflow
Tenant serving	Serverless Inference	Cloud Run, <code>NW_MODEL_URI</code>	managed online endpoint	the course service
Monitoring	Model Monitor on live	Model Monitoring on live	model monitoring on live	Evidently
Retraining	<code>pipeline-submit</code> with <code>ARGS</code>	same	same	same, locally

Acceptance criteria

1. Trainable parameters under 2 percent; the data passed its contract
2. ONNX fp32 within $1e-4$ of PyTorch; int8 smaller; four benchmark rows measured
3. The gate read three files and moved `latest`
4. The pipeline registered one candidate; you approved it with a reason
5. The service serves that version from the registry, int8

32 passed

what `make session03` prints when you are done

Questions worth asking

- Project 2 is slower than Project 1 by a factor you measured. For which tickets is that worth paying?
- The German tickets are five percent and the base model is English. What would you change, and what does it cost per thousand tickets?
- The gate holds int8 to three points against fp32. At what point would you serve fp32, and who pays for the latency?
- If the customer's stack is TensorFlow Serving, what in this part transfers unchanged?